



3f Grunt: New Position Manager Fee Strategy

Security Review

Cantina Managed review by:

M4rio.eth, Lead Security Researcher

Chinmay Farkya, Security Researcher

May 28, 2026

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
3	Findings	4
3.1	Informational	4
3.1.1	Fee documentation comments still describe stale fee formula and snapshot fields . .	4
3.1.2	Recovery from an all-bad-debt fee checkpoint skips the first performance fee	4
3.1.3	Management fee basis has a cliff when a module crosses into bad debt	4
3.1.4	<code>totalDebt</code> is rounded down because it is built from <code>totalBorrowed()</code>	5

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

From May 27th to May 28th the Cantina team conducted a review of [grunt](#) on commit hash [76000e91](#) (PR [190](#)). The team identified a total of **4** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	0	0	0
Gas Optimizations	0	0	0
Informational	4	4	0
Total	4	4	0

The Cantina Managed team reviewed 3F's [grunt](#) holistically on commit hash [a8928137](#) (PR [193](#)) and concluded that all findings were addressed and no new vulnerabilities were identified.

3 Findings

3.1 Informational

3.1.1 Fee documentation comments still describe stale fee formula and snapshot fields

Severity: Informational

Context: `IPositionManagerAdmin.sol#L101-L102`, `PositionManagerBase.sol#L53`, `LibStorage.sol#L19`, `PositionManagerAdmin.sol#L29`, `PositionManagerAdmin.sol#L58`

Description: Several NatSpec and inline comments for the new fee implementation still describe the previous fee model or an incomplete snapshot update. `setFeeData()` still says performance fees are charged on net gains after management-fee deduction, while the rewritten implementation charges performance fees on the levered-slice basis. Two formula comments document `mulDiv(lastDebt, currentCollat, lastCollat)`, but the code intentionally uses `mulDivUp(...)`. The borrow-module admin comments also say only `lastTotalAssets` is updated, while `updateSnapshot()` refreshes both `lastTotalAssets` and `lastDebt`.

Recommendation: Update the affected comments so they match the implementation. `setFeeData()` should describe the levered-slice performance-fee basis after subtracting management-fee assets. The simplified formula comments should use `mulDivUp(lastDebt, currentCollat, lastCollat) - currentDebt`. The add/remove borrow-module comments should state that both `lastTotalAssets` and `lastDebt` are refreshed.

3F: Fixed in PR 193.

Cantina Managed: Verified.

3.1.2 Recovery from an all-bad-debt fee checkpoint skips the first performance fee

Severity: Informational

Context: `LibView.sol#L51-L60`, `LibView.sol#L69-L74`, `PositionManagerBase.sol#L111-L121`, `PositionManagerBase.sol#L181-L185`

Description: Fee accounting excludes borrow modules whose debt exceeds their quoted collateral. If all borrow modules are in bad debt when fees are accrued, the fee snapshot records `lastTotalAssets == 0` and `lastDebt == 0`. The performance-fee branch also treats `lastDebt == 0` as the bootstrap sentinel, so if the position later recovers above water before liquidation, the first recovery accrual skips the performance fee and only seeds the recovered debt baseline.

Recommendation: Document this as an accepted fee-accounting edge case. If this behavior is not intended, separate the bootstrap state from an all-bad-debt checkpoint, for example by using an explicit initialization flag instead of overloading `lastDebt == 0`.

3F: Fixed in PR 193.

Cantina Managed: Verified.

3.1.3 Management fee basis has a cliff when a module crosses into bad debt

Severity: Informational

Context: `LibView.sol#L51-L60`, `LibView.sol#L69-L74`, `PositionManagerBase.sol#L103-L108`

Description: Management fees are charged on the aggregate quoted collateral of borrow modules that are not in bad debt. A module is included while `collateral >= debt`, and excluded entirely once `debt > collateral`. This creates a sharp cliff at the bad-debt boundary: a module at exactly 100% LTV contributes its full quoted collateral to the management-fee basis, while the same module one wei further underwater contributes zero. The vault NAV is effectively unchanged at that boundary, but the management-fee basis can drop by the module's full collateral amount.

Recommendation: Document that bad-debt modules contribute zero collateral to the management-fee basis, even if they are only slightly underwater. If fee undercollection at this boundary is not intended,

change the fee basis to account for the elapsed-period collateral before bad-debt exclusion, or use a smoother accounting rule for modules near the bad-debt threshold.

3F: Fixed in [PR 193](#).

Cantina Managed: Verified.

3.1.4 `totalDebt` is rounded down because it is built from `totalBorrowed()`

Severity: Informational

Context: [MorphoBorrowPosition.sol#L450-L469](#), [LibView.sol#L40-L47](#), [LibView.sol#L69-L74](#), [LibStorage.sol#L152-L162](#), [PositionManager.sol#L177-L180](#), [IPositionManager.sol#L84-L89](#)

Description: `LibView.totalAssets()` returns `totalDebt` by summing each module's `totalBorrowed()` value. For Morpho positions, `totalBorrowed()` converts borrow shares to assets with `toAssetsDown`, intentionally rounding debt down so `repay(totalBorrowed())` does not revert. Therefore, `totalDebt` is a downward-rounded accounting value rather than an exact or conservative debt amount. `lastDebt` inherits this behavior because `updateSnapshot()` stores the `newDebt` value returned by `totalAssets()`, but the primary issue is that `totalDebt` itself is sourced from a rounded-down debt view. The current fee formula is internally consistent because both snapshot debt and current debt use the same path, but future code should not treat these values as exact total debt.

Recommendation: Document `totalDebt` as a fee-accounting debt value sourced from `totalBorrowed()` and therefore rounded down for Morpho modules. Also document that `lastDebt` inherits the same rounding because it snapshots `totalDebt`. If future logic needs exact or conservatively rounded-up debt, add a separately named view or snapshot and compute it with the appropriate rounding direction. Avoid reusing `totalDebt` or `lastDebt` for solvency, reporting, or fee logic that expects exact debt.

3F: Fixed in [PR 193](#).

Cantina Managed: Verified.